Sam wants your RAM.

We're asking whose laptop isn't using theirs.

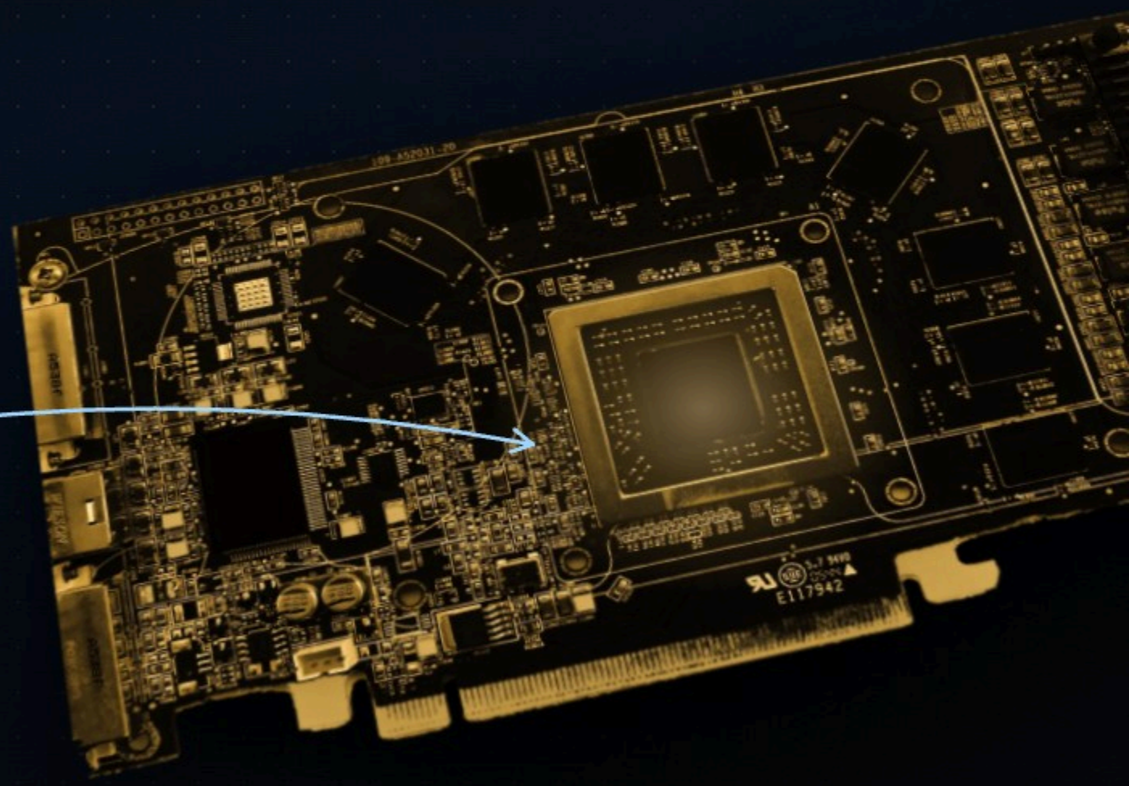
+ NEAR HACKATHON 2026 · PITCH

RAMX.

Spare GPUs. One bigger model.

Pool the laptops people already own, and run AI models that none of them can run alone.

your idle GPU,
tonight



GPU photo: Trio3D / Wikimedia Commons, CC BY-SA 3.0 (edited)

You want to run a big AI model.

```
$ run qwen3-30b-a3b --8bit
› 48 layers · ≈ 32.5 GB of weights
› this laptop: 16 GB memory · ≈ 8 GB usable
› loading weights [▓▓▓
```

Most consumer GPUs sit idle.



Their owners already paid for them. They earn nothing.

MacBook Air · 16 GB · MacBook · 16 GB · Gaming laptop · RTX 4060 · 8 GB — our own team's laptops, idle every night

Big models live in someone else's data centre.



Expensive

You rent GPUs by the hour or pay per token.
The bill grows with every question.

1→N

Centralised

A few providers own the hardware and decide the price, the models and who gets access.

⇢ ?

Not private

Your prompts leave your machine and go to a third party you have to trust.

every prompt leaves your machine

Meanwhile, the hardware people already own **stays switched off**.

One laptop: too small.

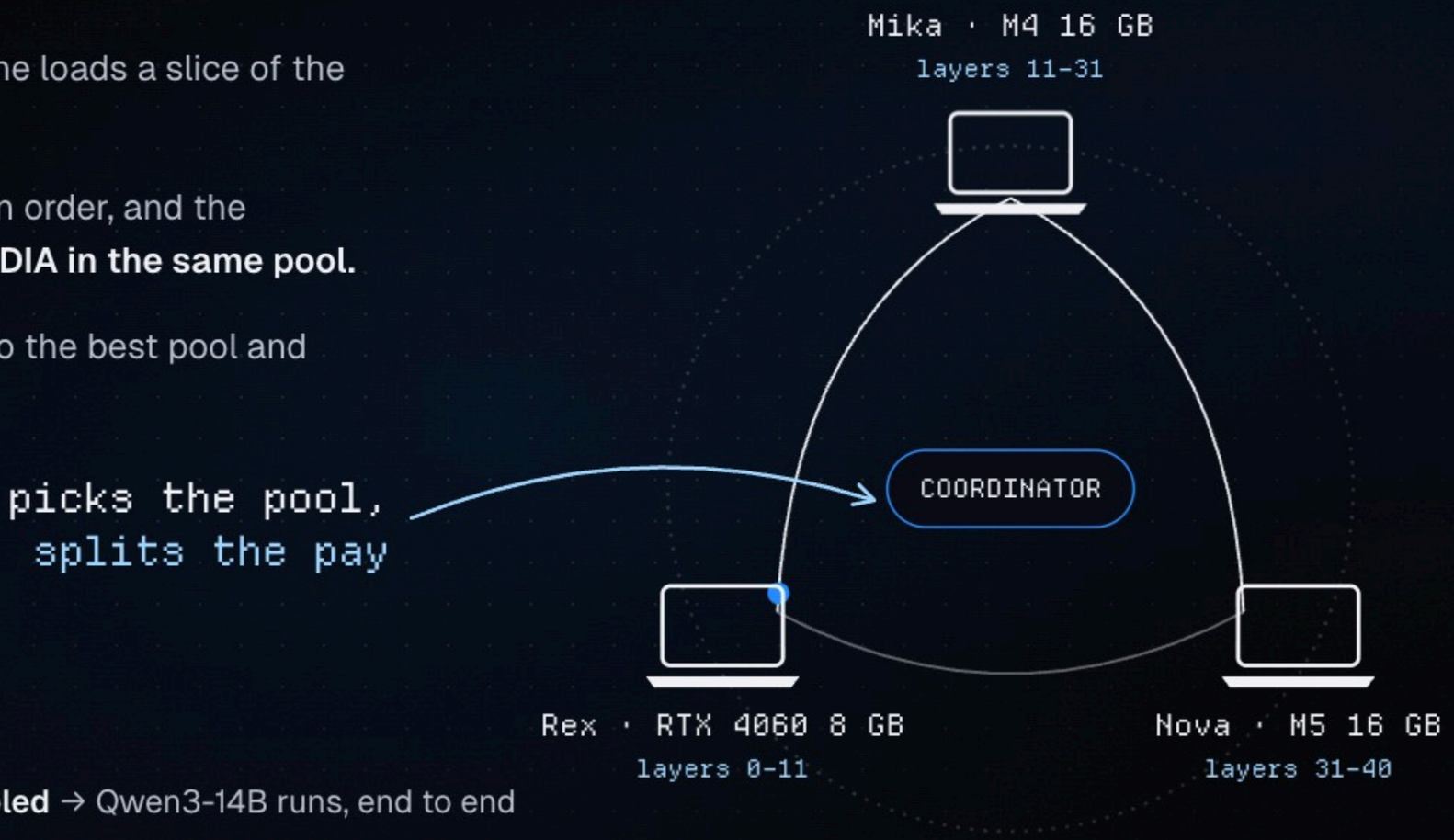
A pool: just right.

Pool the laptops. Split the model.

Nearby laptops form a **pool**. Each one loads a slice of the model's layers.

A request flows through the slices in order, and the answer streams back. **Mac and NVIDIA in the same pool.**

A coordinator routes each request to the best pool and **splits the payment by compute**.



`16 GB` + `8 GB` + `16 GB` = **40 GB pooled** → Qwen3-14B runs, end to end

Every token paid for, on the record.

you pay only this

1 · HOLD
Maximum cost reserved
3,728.2 cr

→

2 · SERVE
The pool streams the answer
256 tokens

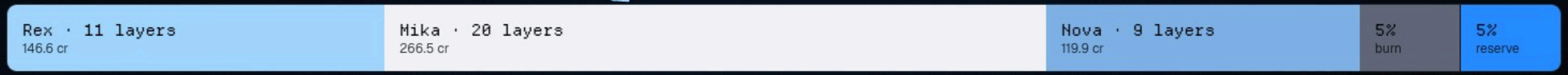
→

3 · SETTLE
Only real usage is charged
592.2 cr

→

4 · RECEIPT
Signed, on the NEAR
ledger

90% back to the laptops



90% to the laptops, split by the layers each one served · example: Qwen3-14B, 412 tokens in, 256 out, model factor 1.75

No account? AI agents pay per request with x402 in USDC on NEAR

Providers claim USDC each epoch with a Merkle proof

Guarded agents pay through an escrow contract

No credits for serving yourself

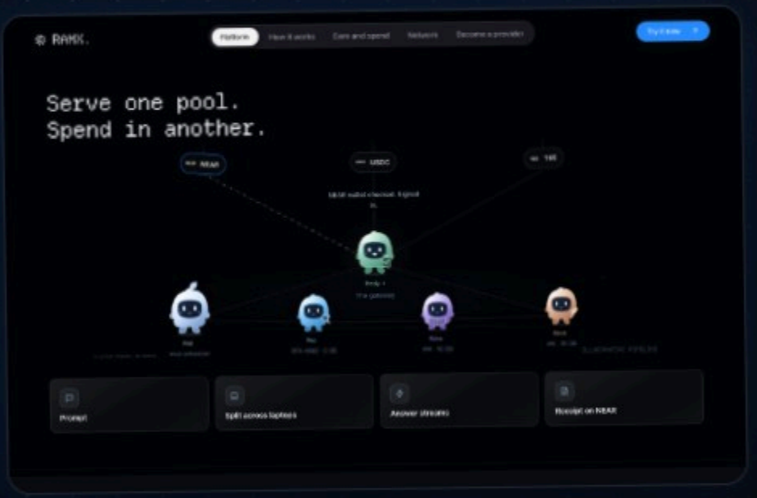
A real product, not a mockup.



landing · the GPU you already own



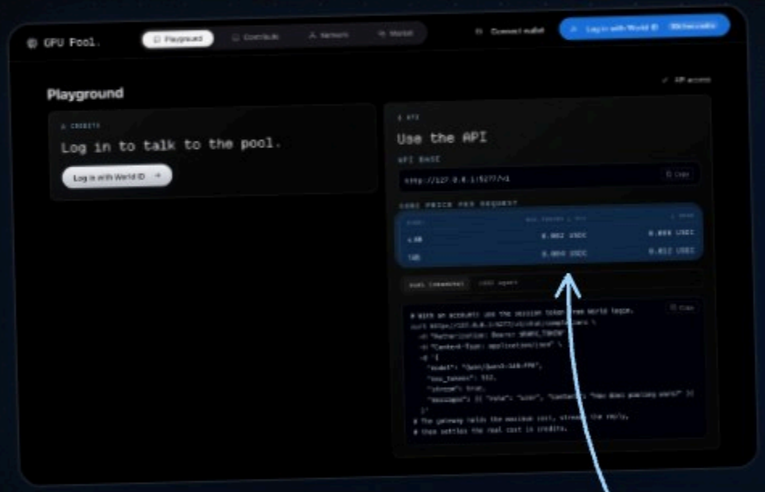
how it works · scroll story, query to payout



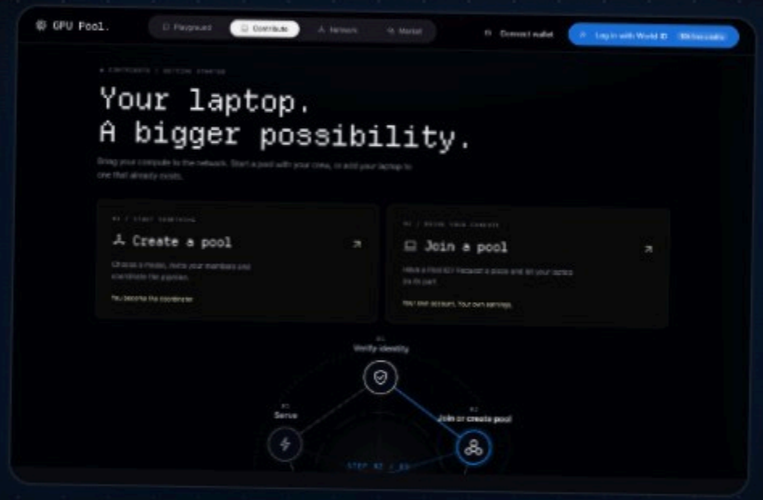
serve one pool · spend in another

the real site, running

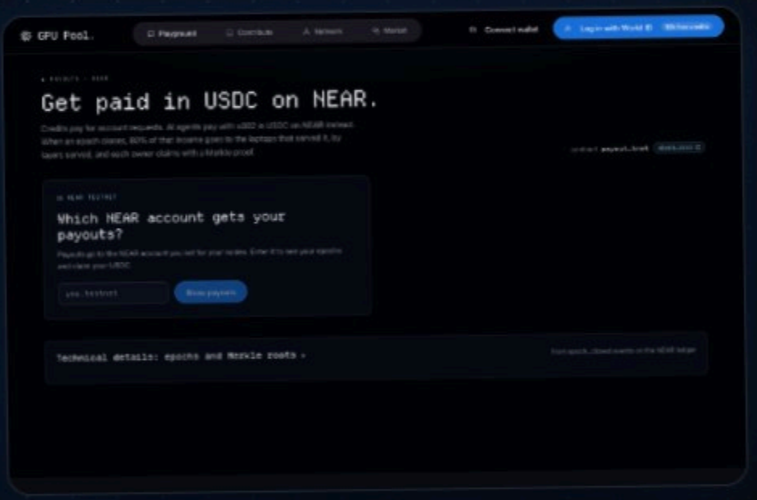
Chat, contribute, get paid.



playground · API and x402 price per request



contribute · create or join a pool



payouts · claim USDC with a Merkle proof

x402 price per request

One request, end to end.

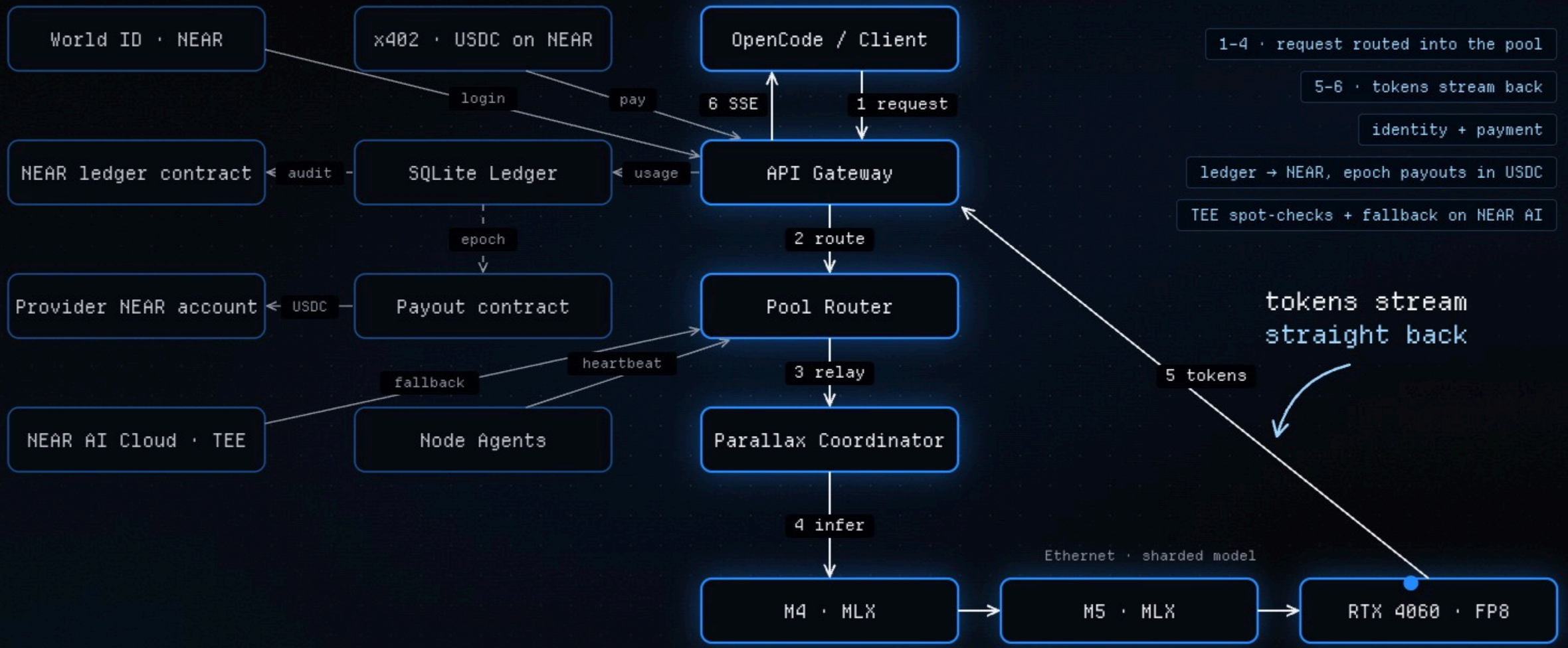


Diagram: Yash, RAMX team

A private pool, from one command.

01 **Log in with World ID or a NEAR wallet**
World ID: one human, one account. NEAR: the wallet signs a login message, no fee.

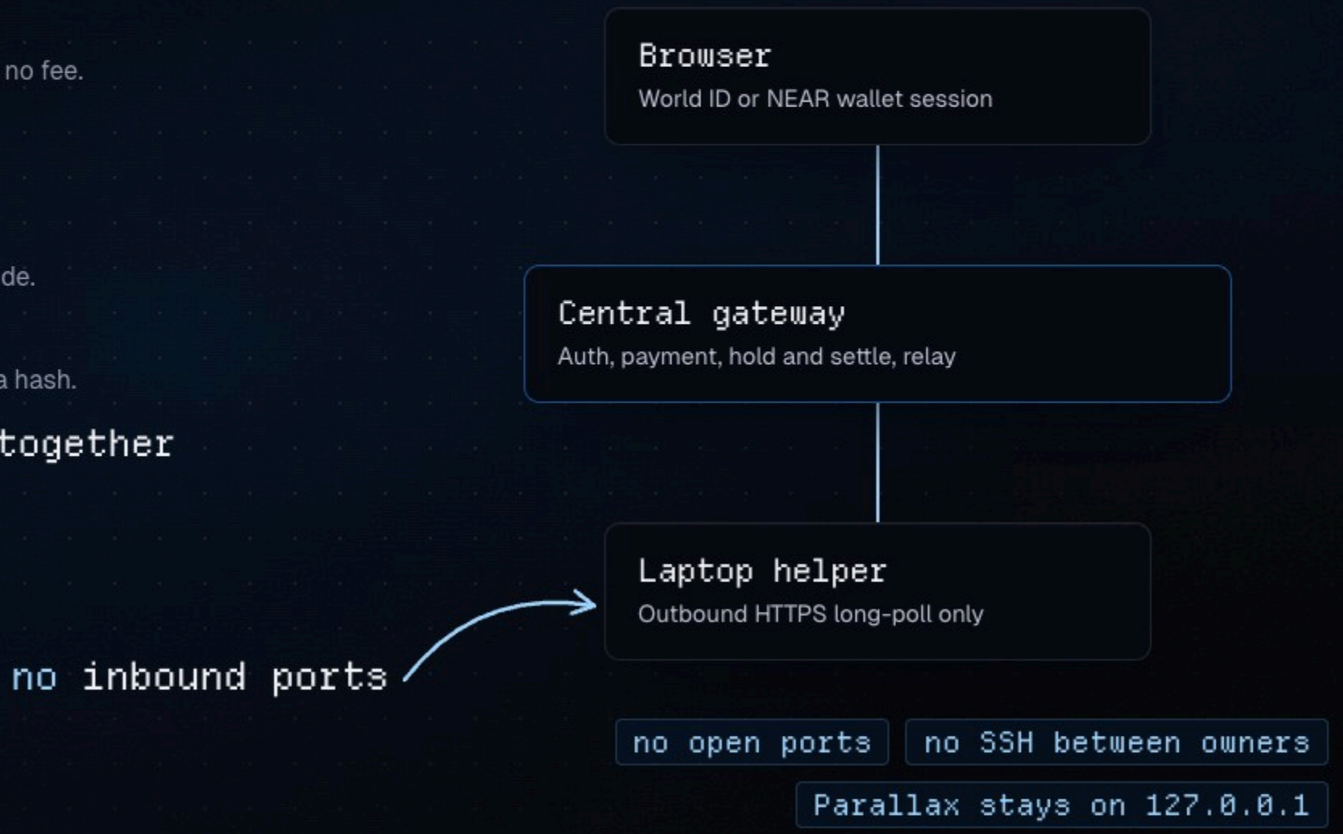
02 **Create a pool, or ask to join**
Knowing a Pool ID is not enough: the coordinator approves every member.

03 **Paste one install command**
A single-use pairing code that expires in 15 minutes. No session or keys inside.

04 **The helper makes its own node key**
It signs its registration and gets a scoped credential. The server keeps only a hash.

05 **Models download first, everyone starts together**
The gateway issues one start time. Nobody pastes timed join commands.

06 **Live in the directory, ready to route**
Readiness needs signed heartbeats and full layer coverage.



The best pool, for every request.

score = reliability × (1 − load)

reliability = 0.7 × success + 0.3 × uptime

The pool is the reputation. Requests it served over 7 days, plus heartbeat slots where every layer was covered. A new pool starts at 0.8.

load = running ÷ slots

Pools batch requests (2 at once today), so a pool at 1 of 2 slots is 50% loaded and still a candidate.



memory pressure lowers a score, never drops the only pool

mid-stream failover continues the same answer

private pools and pinning

Measured on real laptops.

88% less swap

~~2.7~~ → 0.32

GiB swapped per run

A smaller KV cache stopped macOS paging weights to disk on 16 GB Macs: 88% less swap.

~~3.2~~ → 6.0

tokens per second

Same fix, Qwen3-14B across a MacBook Air and an RTX 4060 laptop: up to 85% faster.

0.65 s

to the first token

Measured 0.65–1.2 s after the fix, with stalls down from 12 to 2 per run.

+ Apple MLX and NVIDIA CUDA serve one model together
+ Laptops less than 5 ms apart form pools automatically
+ Ghost-node guard: stop by process group, nothing left behind
+ Mid-request failover, stall detection, clean restarts

Safe for strangers, by design.



World

One human, one account. Starter credits go to real people, and a banned person can't come back as someone new.

IDKit | Orb or Selfie Check



NEAR

Every balance change is signed and written to a public ledger contract. Agents pay per request in USDC with x402. Providers claim USDC with a Merkle proof. Guarded agents can only pay through escrow.

ledger contract | x402 · USDC | payout | escrow



NEAR AI

A TEE model spot-checks 10% of pool answers with a fresh attestation. When no laptop pool is ready, NEAR AI Cloud serves the request, attestation on the receipt.

TEE judge | attested fallback

More AI from hardware that already exists.

14B

model served by a **16 GB Mac + an 8 GB laptop**. Neither can run it alone. MEASURED

90%

of every payment goes to **the people who lent the compute**, by the layers they served. CLAIMED IN USDC ON NEAR TESTNET

0

new GPUs bought. Every pool runs on laptops that would otherwise sit idle.

ESTIMATE · SIMPLE MATHS, NOT A MEASUREMENT

3 laptops (16 + 8 + 16 GB)

= 40 GB

of pooled memory: about the memory of one 40 GB data-centre GPU, assembled from hardware people already own.

100 pools like it

≈ 4 TB

of model memory without building a new data centre.

simple maths, not a measurement

Lend a laptop.
Use the whole network.

World · NEAR · NEAR AI

RAMX · NEAR Hackathon 2026 · **github.com/yashhhhapexxxx/RAMX**